startTests.bat – Quick User Manual

## startTests.bat – AGATE Kommandozeilen-Einstiegspunkt

`startTests.bat` ist der zentrale Kommandozeilen-Einstiegspunkt für den AGATE Server.

Über dasselbe Script können fünf typische Aufgaben ausgeführt werden:

1. Test Suite ausführen
2. Template + Testdaten instanziieren
3. YAML validieren
4. Report analysieren
5. AGATE DSL anzeigen

---

## 1. Test Suite ausführen

### Syntax

```
startTests.bat <user> <instance> <app> <testSuite> [testCase]
[priority]
```

Beispiel:

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Bedeutung:

| Parameter | Beispiel | Beschreibung |
|---|---|---|
| user | Testuser | Benutzer, unter dem der Test ausgeführt wird |
| instance | ACME | Testumgebung / Konfigurationsinstanz |
| app | TOSCATOAGATE | AGATE Application |
| testSuite | Instance_AGATE_Demo.yaml | Auszuführende Test Suite |
| testCase | optional | Nur einen bestimmten TestCase ausführen |
| priority | optional | Tests nach Priorität filtern |

Der Aufruf:

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

bedeutet somit:

```
User        = Testuser
Instance    = ACME
Application = TOSCATOAGATE
Test Suite  = Instance_AGATE_Demo.yaml
```

Die Test Suite wird nicht über einen hart codierten `data`-Pfad gesucht.

AGATE verwendet den in `project-agate.yaml` definierten `paths.tests`-Pfad und ergänzt automatisch Application und Dateinamen:

```
<paths.tests>\<app>\<testSuite>
```

Beispiel:

```
paths:
  tests: 'C:\work\projects\agate-studio-new\agate-
server\ecs_fach\datax'
```

und:

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

ergibt intern:

```
C:\work\projects\agate-studio-new\agate-
server\ecs_fach\datax\toscatoagate\Instance_AGATE_Demo.yaml
```

---

### 1.1. Einen bestimmten TestCase ausführen

Optional kann ein einzelner TestCase angegeben werden.

Beispiel:

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
AGATE_Demo
```

Damit wird nur:

```
AGATE_Demo
```

aus der angegebenen Test Suite ausgeführt.

---

### 1.2. TestCase und Priority angeben

Zusätzlich kann die Priorität angegeben werden:

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
AGATE_Demo HIGH
```

Damit kann die Testausführung zusätzlich auf die gewünschte Priorität eingeschränkt werden.

## 2. AGATE Project-Konfiguration

AGATE verwendet eine zentrale Projektdatei:

```
project-agate.yaml
```

Beispiel:

```
version: 1

project:
  name: fach

paths:
  tests: 'C:\work\projects\agate-studio-new\agate-server\ecs_fach\datax'
  responses: 'C:\work\projects\agate-studio-new\agate-server\ecs_fach\responses'
  reports: 'C:\work\projects\agate-studio-new\agate-server\ecs_fach\report'

config:
  environments: 'C:\work\projects\agate-studio-new\agate-server\ecs_fach\env\env.conf'
  users: 'C:\work\projects\agate-studio-new\agate-server\ecs_fach\env\users.conf'
```

Damit kennt AGATE zentral:

```
Tests
Responses
Reports
Environment-Konfiguration
User-Konfiguration
```

Die CLI-Aufrufe müssen diese Verzeichnisse deshalb nicht mehr vollständig angeben.

---

### 2.1. Konfigurierte Pfade

#### tests

Root-Verzeichnis aller AGATE Applications und Test Suites.

Beispiel:

```
C:\work\projects\agate-studio-new\agate-server\ecs_fach\datax
```

Darunter kann eine Application beispielsweise so aufgebaut sein:

```
datax
└── dmp11agate
    ├── modules
    ├── reusable
    ├── template
    ├── CheckStatus.yaml
    └── Instance_DMP SS12 V11 doAusschreibung.yaml
```

---

#### responses

Zentrales Verzeichnis für gespeicherte REST-/SOAP-Referenzantworten.

Beispiel:

```
C:\work\projects\agate-studio-new\agate-server\ecs_fach\responses
```

---

**reports**

Root-Verzeichnis für erzeugte AGATE Reports.

Beispiel:

```
C:\work\projects\agate-studio-new\agate-server\ecs_fach\report
```

---

**environments**

Environment-Konfiguration:

```
C:\work\projects\agate-studio-new\agate-server\ecs_fach\env\env.conf
```

---

**users**

User-Konfiguration:

```
C:\work\projects\agate-studio-new\agate-server\ecs_fach\env\users.conf
```

---

### 2.2. Relative und absolute Pfade

Die Pfade in `project-agate.yaml` können absolut oder relativ zum AGATE Project Root angegeben werden.

Beispiel absolut:

```
paths:
  tests: 'C:\work\projects\agate-studio-new\agate-
server\ecs_fach\datax'
```

Beispiel relativ:

```
paths:
  tests: 'ecs_fach/datax'
```

---

### 2.3. Project Root

Wenn kein `--project` angegeben wird, verwendet AGATE das aktuelle Working Directory als Project Root.

Beispiel:

```
cd C:\work\projects\agate-studio-new\agate-server
```

danach:

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Optional kann ein Projekt explizit angegeben werden:

```
startTests.bat --project C:\TOSCA_PROJECTS\DMP11 Testuser ACME
DMP11AGATE "Instance_DMP SS12 V11 doAusschreibung.yaml"
```

---

## 3. Konfiguration für die Testausführung

Die beim Start angegebenen Werte:

```
user
instance
```

werden für die AGATE-Konfiguration verwendet.

Für:

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

ist beispielsweise die Instance:

```
ACME
```

---

### 3.1. Environment-Konfiguration

Der Pfad zur Environment-Konfiguration wird über `project-agate.yaml` definiert.

Beispiel:

```
config:
  environments: 'C:\work\projects\agate-studio-new\agate-
server\ecs_fach\env\env.conf'
```

In der Datei können Environment-Werte definiert werden.

Beispiel:

```
# --- TEST INSTANCE: ACME ---
ACME.math.url=http://api.mathjs.org
```

Im Test kann dieser Wert beispielsweise verwendet werden als:

```
endpoint: "{E[env.math.url]}"
```

---

### 3.2. User-Konfiguration

Auch der Pfad zur User-Konfiguration wird zentral über `project-agate.yaml` definiert.

Beispiel:

```
config:
  users: 'C:\work\projects\agate-studio-new\agate-
server\ecs_fach\env\users.conf'
```

Für:

```
Instance = ACME
User     = Testuser
```

erwartet AGATE User-Einträge mit dem Prefix:

```
ACME.Testuser.
```

Beispiel:

```
# --- USER: ACME ---
ACME.Testuser.ip=127.0.0.1
ACME.Testuser.email=testuser@acme.local
```

Eine einzelne Zeile wie:

```
ACME.Testuser=Mustermann
```

reicht nicht aus, da AGATE nach Konfigurationseinträgen mit:

```
ACME.Testuser.
```

sucht.

---

## 4. Template instanziieren

Die Instanziierung ist insbesondere für Template-basierte Tests wichtig, beispielsweise für Tests, die aus **Tosca TestCase Design** migriert wurden.

Dabei existieren zwei getrennte Artefakte:

```
Template YAML
+
CSV Testdaten
```

Beispiel:

```
AGATE_Demo.yaml
TCD_AGATE_Demo.csv
```

Diese beiden Dateien sind noch nicht die endgültige ausführbare Test Suite.

Mit `instantiate` werden Template und Testdaten kombiniert und daraus konkrete AGATE TestCases erzeugt.

---

### 4.1. Syntax

```
startTests.bat instantiate <appName> <templateFile> <dataFile>
```

Beispiel:

```
startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
TCD_AGATE_Demo.csv
```

Dabei gilt:

| Parameter | Beispiel | Bedeutung |
|---|---|---|

| | | |
|---|---|---|
| appName | TOSCATOAGATE | Application, in deren `template`-Verzeichnis die Dateien liegen |
| templateFile | AGATE_Demo.yaml | YAML Template mit der Testlogik |
| dataFile | TCD_AGATE_Demo.csv | CSV mit den konkreten Testdaten |

AGATE ermittelt die vollständigen Pfade automatisch über `project-agate.yaml`.

Intern werden daraus:

```
<paths.tests>\<appName>\template\<templateFile>
```

und:

```
<paths.tests>\<appName>\template\<dataFile>
```

Der Benutzer muss also keinen vollständigen Dateipfad mehr angeben.

---

### 4.2. Was passiert bei instantiate?

Das Template enthält die Testlogik, aber bestimmte Werte werden nicht direkt im YAML hinterlegt.

Beispiel:

```
- type: BUFFER
  op: EXEC
  name: 'x'
  value: '{XL[x]}'

- type: BUFFER
  op: EXEC
  name: 'y'
  value: '{XL[y]}'

- type: BUFFER
  op: EXEC
  name: 'z'
  value: '{XL[z]}'
```

Die Ausdrücke:

```
{XL[x]}
{XL[y]}
{XL[z]}
```

verweisen auf Werte aus der CSV-Datei.

Beispielsweise kann die CSV unterschiedliche Datensätze enthalten:

```
x ; y ; z
```

```
2 ; 7 ; 3
22 ; 70 ; 3
```

Das Template definiert somit:

```
WAS getestet wird
```

und die CSV definiert:

```
MIT WELCHEN DATEN getestet wird
```

`instantiate` kombiniert beides.

---

### 4.3. Ablauf der Instanziierung

Der Ablauf kann vereinfacht so dargestellt werden:

```
AGATE_Demo.yaml
      +
TCD_AGATE_Demo.csv
      |
      v
startTests.bat instantiate
      |
      v
Template wird gelesen
      |
      v
CSV-Datensätze werden gelesen
      |
      v
{XL[...]} werden mit konkreten Werten aufgelöst
      |
      v
konkrete TestCases werden erzeugt
      |
      v
ausführbare AGATE Test Suite
```

Das Template enthält beispielsweise:

```
name: 'x'
value: '{XL[x]}'
```

und der CSV-Datensatz enthält:

```
x = 2
```

Dann erhält der konkrete TestCase für diesen Datensatz:

```
x = 2
```

Beim nächsten CSV-Datensatz könnte dagegen gelten:

```
x = 22
```

Die Testlogik bleibt identisch.

Nur die Testdaten ändern sich.

---

### 4.4. Warum gibt es Template und Instanz getrennt?

Der Vorteil ist, dass eine Testlogik nur einmal beschrieben werden muss.

```
                    AGATE_Demo.yaml
                          |
              +-----------+-----------+
              |                       |
              v                       v
         Datensatz 1             Datensatz 2
         x=2, y=7, z=3           x=22, y=70, z=3
              |                       |
              v                       v
          TestCase 1              TestCase 2
```

Ohne Templates müsste für jeden Datensatz eine eigene YAML-Testdefinition gepflegt werden.

Mit Template + CSV kann dieselbe Testlogik für viele Datenkombinationen verwendet werden.

---

### 4.5. Application-Struktur

Die konkrete Verzeichnisstruktur wird über `paths.tests` bestimmt.

Beispiel:

```
<paths.tests>
└── TOSCATOAGATE
    ├── reusable
    ├── modules
    ├── template
    │   ├── AGATE_Demo.yaml
    │   └── TCD_AGATE_Demo.csv
    └── Instance_AGATE_Demo.yaml
```

Die Template-Dateien werden instanziiert mit:

```
startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
TCD_AGATE_Demo.csv
```

Das Ergebnis wird direkt im Application-Verzeichnis erzeugt:

```
<paths.tests>\TOSCATOAGATE\Instance_AGATE_Demo.yaml
```

---

### 4.6. Instanziierten Test ausführen

Nach erfolgreicher Instanziierung wird der erzeugte Test genau wie jede andere AGATE Test Suite ausgeführt.

Beispiel:

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Der vollständige Workflow ist:

```
AGATE_Demo.yaml
      +
TCD_AGATE_Demo.csv
      |
      v
```

```
INSTANTIATE
      |
      v
Instance_AGATE_Demo.yaml
      |
      v
VALIDATE
      |
      v
EXECUTE
```

---

### 4.7. Empfohlener Workflow für Template-Tests

**Schritt 1 – Template und CSV prüfen**

```
<paths.tests>\TOSCATOAGATE\template\AGATE_Demo.yaml
<paths.tests>\TOSCATOAGATE\template\TCD_AGATE_Demo.csv
```

**Schritt 2 – Instanziieren**

```
startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
TCD_AGATE_Demo.csv
```

**Schritt 3 – Generiertes YAML validieren**

```
startTests.bat validate TOSCATOAGATE Instance_AGATE_Demo.yaml
```

**Schritt 4 – Test ausführen**

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Damit ist der gesamte Template-Workflow:

```
Template
   +
Testdaten
   |
   v
INSTANTIATE
   |
   v
konkrete Test Suite
   |
   v
VALIDATE
   |
   v
EXECUTE
```

---

## 5. YAML validieren

Mit `validate` kann eine AGATE YAML-Datei geprüft werden, ohne den eigentlichen Test auszuführen.

### Syntax

```
startTests.bat validate <appName> <yamlFile>
```

Beispiel:

```
startTests.bat validate TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Weiteres Beispiel:

```
startTests.bat validate MUHI CheckStatus.yaml
```

Bei Dateinamen mit Leerzeichen:

```
startTests.bat validate DMP11AGATE "Instance_DMP SS12 V11
doAusschreibung.yaml"
```

Der vollständige Pfad muss nicht angegeben werden.

AGATE bildet ihn automatisch aus:

```
<paths.tests>\<appName>\<yamlFile>
```

Beispiel:

```
paths:
  tests: 'C:\work\projects\agate-studio-new\agate-
server\ecs_fach\datax'
```

Aufruf:

```
startTests.bat validate DMP11AGATE "Instance_DMP SS12 V11
doAusschreibung.yaml"
```

ergibt intern:

```
C:\work\projects\agate-studio-new\agate-
server\ecs_fach\datax\dmp11agate\Instance_DMP SS12 V11
doAusschreibung.yaml
```

---

### 5.1. Erfolgreiche Validierung

Bei Erfolg erscheint:

```
======================================================================
            VALIDATION SUCCESSFUL
======================================================================
```

Die Datei kann anschließend ausgeführt werden.

---

### 5.2. Fehlgeschlagene Validierung

Bei Fehlern erscheint beispielsweise:

```
======================================================================
            VALIDATION FAILED
======================================================================
```

Zusätzlich liefert das Script einen Fehler-Exit-Code zurück.

Dies ist insbesondere auch für CI/CD-Pipelines nützlich.

---

### 5.3. Data-Flow-Validierung

Der Validator prüft unter anderem:

```
Buffer-Verwendung vor Initialisierung
Response-Verwendung vor Erzeugung
CALL / Reusable Datenfluss
LOOP Datenfluss
```

Beispiele für typische Fehler:

```
AGATE-V401
Variable/buffer is referenced before initialization
```

oder:

```
AGATE-V402
Response is used before it is created
```

---

### 5.4. LOOP-Semantik

Bei LOOP-Strukturen muss die tatsächliche Ausführungsreihenfolge berücksichtigt werden.

#### WHILE

```
condition
   |
   v
body
   |
   +----> condition
```

Die Condition wird also geprüft, bevor der Body erstmals ausgeführt wird.

Ein Buffer, der ausschließlich im Body erzeugt wird, darf deshalb nicht bereits in der ersten `WHILE`-Condition benötigt werden.

---

#### DO_WHILE

```
body
 |
 v
condition
 |
 +----> body
```

Beim `DO_WHILE` wird der Body dagegen mindestens einmal ausgeführt, bevor die Condition geprüft wird.

Damit ist beispielsweise folgende Konstruktion gültig:

```yaml
- type: LOOP
  mode: DO_WHILE
  condition: "{B[patient_registered_count]} == 0"
  maxIterations: 30
  timeoutMs: 120000

  steps:

    - type: SQL
      op: EXEC
      command: >
        SELECT COUNT(*)
        FROM DMP_BETREUUNGSVERHAELTNIS
      response: patient_registered_response
```

```
15
16      - type: SQL
17        op: BUFFER
18        response: patient_registered_response
19        row: "0"
20        column: "0"
21        name: patient_registered_count
```

Der Ablauf ist:

```
SQL EXEC
   |
   v
SQL BUFFER
   |
   v
patient_registered_count wird gesetzt
   |
   v
DO_WHILE Condition wird geprüft
```

`patient_registered_count` ist somit bei der Condition bereits vorhanden.

---

### 5.5. Empfehlung

Neue oder geänderte Tests sollten vor der Ausführung grundsätzlich validiert werden:

```
YAML
 |
 v
VALIDATE
 |
 +---- Fehler ----> korrigieren
 |
 v
EXECUTE
```

---

## 6. Report analysieren

Mit `analyze` kann ein bereits erzeugter AGATE JSON Report analysiert werden.

### Syntax

```
startTests.bat analyze <jsonReport>
```

Beispiel:

```
startTests.bat analyze
"ecs_fach\report\ECS_SYST_AUT1\dmp11agate\Instance_DMP SS12 V11
doAusschreibung\Latest_Report.json"
```

Der Analyzer:

```
JSON Report
    |
    v
JsonReportAnalyzer
    |
    v
AnalysisResult
    |
    +----> Console-Ausgabe
    |
```

```
    +----> Analysis JSON
```

Für:

```
Latest_Report.json
```

wird beispielsweise erzeugt:

```
Latest_Report_analysis.json
```

Die Analysis-Datei wird im selben Verzeichnis wie der ursprüngliche JSON Report erzeugt.

---

## 7. AGATE DSL anzeigen

Mit `describe` kann die eingebaute AGATE DSL-Dokumentation direkt über die Kommandozeile angezeigt werden.

### Alle verfügbaren DSL-Typen

```
startTests.bat describe
```

Damit erhält man eine Übersicht der verfügbaren AGATE Step Types.

---

### Einen bestimmten DSL-Typ anzeigen

REST:

```
startTests.bat describe REST
```

SOAP:

```
startTests.bat describe SOAP
```

Reusable / CALL:

```
startTests.bat describe CALL
```

Weitere Beispiele:

```
startTests.bat describe SQL
startTests.bat describe BUFFER
startTests.bat describe CMD
startTests.bat describe FILE
startTests.bat describe WAIT
```

`describe` ist damit die schnellste Referenz beim Schreiben eines AGATE Tests.

---

## 8. Typischer Workflow für einen normalen Test

Für eine bereits fertige Test Suite:

```
YAML
 |
 v
VALIDATE
 |
 v
```

```
EXECUTE
 |
 v
ANALYZE
```

Beispiel:

```
startTests.bat validate TOSCATOAGATE Instance_AGATE_Demo.yaml
```

danach:

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Optional anschließend:

```
startTests.bat analyze "<reportPath>\Latest_Report.json"
```

---

## 9. Typischer Workflow für einen Template-Test

Bei Template-basierten Tests kommt vor der Validierung die Instanziierung:

```
Template YAML
     +
CSV Testdaten
     |
     v
INSTANTIATE
     |
     v
konkrete Test Suite
     |
     v
VALIDATE
     |
     v
EXECUTE
     |
     v
ANALYZE
```

Konkretes Beispiel:

```
startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
TCD_AGATE_Demo.csv
```

danach:

```
startTests.bat validate TOSCATOAGATE Instance_AGATE_Demo.yaml
```

anschließend:

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

und optional:

```
startTests.bat analyze "<reportPath>\Latest_Report.json"
```

---

## 10. Quick Reference

### Test ausführen

```
startTests.bat <user> <instance> <app> <testSuite> [testCase]
[priority]
```

Beispiel:

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

---

### Einzelnen TestCase ausführen

```
startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
AGATE_Demo
```

---

### Template instanziieren

```
startTests.bat instantiate <appName> <templateFile> <dataFile>
```

Beispiel:

```
startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
TCD_AGATE_Demo.csv
```

---

### YAML validieren

```
startTests.bat validate <appName> <yamlFile>
```

Beispiel:

```
startTests.bat validate TOSCATOAGATE Instance_AGATE_Demo.yaml
```

oder:

```
startTests.bat validate DMP11AGATE "Instance_DMP SS12 V11
doAusschreibung.yaml"
```

---

### Report analysieren

```
startTests.bat analyze <jsonReport>
```

Beispiel:

```
startTests.bat analyze
"ecs_fach\report\ECS_SYST_AUT1\dmp11agate\Instance_DMP SS12 V11
doAusschreibung\Latest_Report.json"
```

---

### DSL-Typen anzeigen

```
startTests.bat describe
```

oder:

```
startTests.bat describe REST
startTests.bat describe SOAP
startTests.bat describe CALL
startTests.bat describe SQL
```

## 11. Merksatz

```
project-agate.yaml
    ↓
Wo liegen Tests, Responses, Reports und Config?

describe
    ↓
Wie sieht die AGATE DSL aus?

instantiate
    ↓
Template + CSV -> konkrete Tests

validate
    ↓
Ist das YAML technisch korrekt?

startTests
    ↓
Test wirklich ausführen

analyze
    ↓
Was zeigt der erzeugte JSON Report?
```

Für Template-basierte Tests ist die wichtigste Reihenfolge:

```
INSTANTIATE -> VALIDATE -> EXECUTE -> ANALYZE
```

`instantiate` erzeugt aus einer wiederverwendbaren Testlogik und den dazugehörigen Testdaten eine konkrete Test Suite.

`validate` prüft die erzeugte Suite, ohne sie auszuführen.

Der normale Test Runner führt anschließend die konkrete Test Suite aus.

`analyze` kann danach den erzeugten JSON Report auswerten und zusätzlich eine `*_analysis.json`-Datei erzeugen.